

Task Offloading and Resource Pricing Based on Game Theory in UAV-Assisted Edge Computing

Zhuoyue Chen[✉], Yaozong Yang[✉], Jiajie Xu[✉], Ying Chen[✉], *Senior Member, IEEE*,
and Jiwei Huang[✉], *Senior Member, IEEE*

Abstract—Due to the limited battery capacity and computational resources of mobile devices, computation-intensive tasks generated by mobile devices can be offloaded to edge servers for processing. This paper investigates the multi-user task offloading and resource pricing issues in Autonomous aerial vehicle (AAV)-assisted Multi-Access Edge Computing (MEC) systems. The optimization objectives is optimizing the utility of the server and the utility of the Edge Users (EUs), with decision variables encompassing the offloading strategies of EUs and the pricing strategies of the server. We divide the entire optimization problem into two parts. When optimizing the server's utility, server energy consumption is a crucial metric; hence, in the first part, we formulate the user allocation problem with the goal of minimizing the server's overall energy consumption. Utilizing game theory, we transform the user allocation problem into a multi-user non-cooperative game and prove the existence of a Nash Equilibrium (NE). The Game-based User Allocation (GBUA) algorithm is proposed to obtain the user allocation strategy. After addressing the user allocation problem, we consider the simultaneous optimization of both server and EUs utility. Therefore, in the second part, we model the server and EUs's engagement using the Stackelberg game model and employ backward induction to verify the presence of a Stackelberg Equilibrium (SE). Additionally, we propose the Resource Pricing and Task Offloading (RPATO) algorithm, based on game theory, to obtain the SE solution. Finally, extensive experiments are conducted to validate the effectiveness of the proposed algorithms, and numerous comparative algorithms are tested to prove the advancement and innovation of our proposed algorithms.

Index Terms—Multi-Access Edge Computing (MEC), resource pricing, stackelberg game, task offloading, Autonomous aerial vehicle (AAV).

I. INTRODUCTION

AS INTERNET of Things (IoT) devices become increasingly common and technological progress continues, IoT

opens up numerous application domains in the field of computationally intensive functions, such as intelligent transportation systems, smart healthcare, and online interactive gaming [1], [2]. Yet, due to restrictions in power usage and processing capabilities, existing IoT devices are fundamentally incapable of accommodating such applications [3]. Mobile Cloud Computing (MCC) has emerged as a promising approach to enhance the computational capabilities of devices. By offloading computation tasks to well-endowed cloud data centers for completion, MCC surpasses the constraints on a device's battery capacity and computational resources [4].

However, for some latency-sensitive IoT services, MCC is not ideal due to the additional transmission costs and delays. Fortunately, The emergence Multi-Access Edge Computing (MEC) provides a feasible solution to these latency-sensitive tasks [5]. By deploying MEC servers, Edge Users' (EUs) workloads are able to be shifted to adjacent Edge Servers (ES), thereby enhancing the Quality of Experience (QoE) and greatly decreasing power usage and latency [6]. Nevertheless, traditional MEC deploys the ES on the ground base which may not be applicable to certain communication scenarios, such as urgent assistance, armed forces training, and catastrophe relief [7]. In such situations, many terrestrial facilities are damaged, thereby being unable to furnish connectivity and computational support.

Autonomous aerial vehicles (AAVs) are able to utilized to support the networking and processing of MEC, having emerged as a pivotal technology handling the difficulty [8]. UAVs possess superior maneuverability and flexibility, enabling them to be deployed agilely in such environments, providing computing services to EUs within the affected areas. Moreover, UAVs can offer improved channel conditions while airborne, extending the coverage of wireless networks and simultaneously reducing latency, enhancing the speed and quality of data transmission. Furthermore, UAVs are equipped to actively change their positions according to live demands, balancing network loads and optimizing resource allocation, thereby enhancing the overall efficiency and performance of the multi-access edge computing network.

Although many efforts have been made to solve the problem of task offloading in UAV-MEC system [9], [10], there are still some challenges that need to be addressed. First, the addition of UAVs provides EUs with more offloading options. However, it is challenging to make offloading decisions between Base Station (BS)-ES and UAV-ES to achieve load balancing. Second, from an economic perspective, BS-ES and UAV-ES require profits to drive their service provision, while also consuming their

Received 14 October 2024; accepted 25 November 2024. Date of publication 9 December 2024; date of current version 6 February 2025. This work was supported in part by National Natural Science Foundation of China under Grant 62472039, in part by Beijing Natural Science Foundation under Grant L232050, in part by the Project of Cultivation for young top-motch Talents of Beijing Municipal Institutions under Grant BPHR202203225, and in part by Young Elite Scientists Sponsorship Program by BAST under Grant BYESS2023031. (Corresponding author: Ying Chen.)

Zhuoyue Chen, Yaozong Yang, Jiajie Xu, and Ying Chen are with the School of Computer Science, Beijing Information Science and Technology University, Beijing 100101, China (e-mail: chenzyuoyue@bistu.edu.cn; yyz@bistu.edu.cn; xujiajie@bistu.edu.cn; chenying@bistu.edu.cn).

Jiwei Huang is with the Hainan Institute of China University of Petroleum (Beijing) and the Beijing Key Laboratory of Petroleum Data Mining, China University of Petroleum, Beijing 102249, China (e-mail: huangjw@cup.edu.cn).

Digital Object Identifier 10.1109/TSC.2024.3512936

computing and communication resources. We need to make appropriate strategies to balance profits and costs. In addition, the increasing number of IoT devices and network scale, will also bring more challenges, and we need to consider the scalability of algorithms to adapt to different environments.

Stackelberg games can be employed to investigate the competition and cooperation among rational individual participants, wherein the participants are categorized as leaders and followers. However, to the best of our knowledge, the design of Stackelberg games in UAV-assisted MEC systems has not been thoroughly explored. Zhou et al. in [11] utilized Stackelberg games to address the task offloading and resource pricing issues between servers and users, but did not fully consider the utility of both servers and users. In the study by Yuan et al. in [12], a three-level Stackelberg model was proposed to address the optimal resource allocation problem; however, the issue of user allocation was not considered. Xu et al. in [13] focused on resolving anti-interference transmission issues in communication networks through Stackelberg games, but did not address the utility issue between users and servers. Yang et al. in [14] employed Stackelberg games to study the interaction between UAVs and attackers, but did not consider the overall utility of the system. Therefore, the scenario we consider involves EUs being assigned tasks for execution, with both BS-ES and UAV-ES possessing unused computational capacity. EUs may choose to delegate tasks to the BS-ES or UAV-ES to alleviate pressure and costs, while the servers can profit from monetizing their unused computing power [15]. Specifically, we apply game-theoretic approaches to address the user allocation problem and utilize a Stackelberg game to resolve the task offloading process between servers and EUs, with BS-ES and UAV-ES acting as leaders and EUs as followers. Subsequently, we introduce an algorithm to obtain a relatively optimal solution that maximizes the utility for both servers and EUs. The primary contributions can be summarized as follows:

- We consider the scenario of UAV-assisted MEC where there exist several EUs, one BS-ES, and one UAV-ES, and investigate the computation task offloading problem of EUs in this scenario. The offloading strategies of EUs and the pricing strategies of servers are considered as decision variables, aiming to optimize the utilities of both servers and EUs.
- We consider the EUs' allocation problem with the objective of optimizing the server's energy consumption. Employing game theory, we transform the user allocation problem into a non-cooperative game among EUs, prove the existence of a Nash Equilibrium (NE), and propose the Game-based User Allocation (GBUA) algorithm to resolve the user allocation problem for EUs.
- We utilize a Stackelberg game model to characterize the server and EUs's engagement, in which the BS-ES and UAV-ES act as leaders, while the EUs act as followers. The optimization objective is to maximize the utility for both servers and EUs. We employ backward induction to analyze the proposed Stackelberg model and prove that there exists a unique Stackelberg Equilibrium (SE) between the servers and EUs. Furthermore, we have designed the Resource Pricing and Task Offloading (RPATO) algorithm to obtain an approximate solution.

- We evaluate the proposed algorithms through extensive experimentation, and the results prove the feasibility and effectiveness of the algorithms. Additionally, a comprehensive set of comparative experiments has been conducted, pitting our methods against several traditional baseline approaches. The experimental outcomes highlight the advancement and innovation of our proposed algorithms.

The structure of the paper is as follows, followed by a discussion of related work in Section II. In Section III, the system model is considered and the problem is formulated. The offloading problem is analyzed using game theory in Section IV. The experimental evaluation is presented in Section V. Section VI concludes the paper.

II. RELATED WORK

A. Computation Offloading in MEC

In recent years, computation offloading in MEC has garnered significant attention from both the academic and industrial sectors, with a multitude of studies focusing on computation offloading in ground-based MEC systems. Guo et al. in [16] considered the communication and computation dynamics between ground BSs and users in MEC systems, aiming to design an online computation task offloading mechanism to optimize task execution latency. Meanwhile, Wang et al. in [17] also considered ground-based MEC systems and utilized the Lyapunov optimization framework to make optimal single-time-slot decisions for user transmission power allocation and MEC server selection, with the goal of minimizing time-averaged expected task execution delay under the constraint of average energy consumption. Dai et al. in [18] considered a new scenario in ground-based MEC systems, treating vehicles as small BSs, and studied the coordination of task uploading among multiple vehicles to construct a collaborative computation offloading problem, aiming to minimize the latency of task completion. Additionally, in [19], the authors proposed a ground vehicular network that supports MEC and introduced a new computation task offloading mechanism designed to minimize energy consumption during task completion. Although task offloading in ground-based MEC systems has been extensively researched, its physical characteristics on the ground make it challenging to conduct studies in more flexible areas.

B. UAVs-Assisted Edge Computing

As ground-based MEC systems continue to develop, they also face challenges in conducting studies in more flexible regions. However, amidst the rapid development of MEC, drones have also found widespread application across various domains. Zhu et al. in [20] proposed a computation offloading scenario in drone-assisted MEC, outlining a new paradigm for drone-assisted computation offloading, and endeavored to design a novel computation task offloading mechanism to optimize task execution latency. Concurrently, Lin et al. in [21] proposed a dynamic resource allocation for drone-assisted MEC based on Lyapunov, aiming to select appropriate servers for users and minimize time-averaged expected task execution delay under the constraint of average energy consumption. Simultaneously, research efforts have been directed towards optimizing energy

consumption within the system. Zhao et al. in [22] studied a collaborative multi-access edge computing system with multiple drones and multiple edge clouds, aiming to reduce the total energy consumption of the system by optimizing the drone's trajectory and the allocation of computation tasks. Lu et al. in [23] explored a drone-assisted MEC scenario and proposed two multi-drone-assisted multi-access edge computing transmission methods based on single-agent and multi-agent reinforcement learning, aiming to minimize the total energy consumption of the system. A plethora of research has been dedicated to optimizing single attributes such as system delay and energy consumption, and the overall performance optimization of the system has gradually become an unavoidable challenge.

C. Game Theory in Task Offloading

When considering the overall performance in MEC systems, game theory is a favorable approach for addressing the issue. Xu et al. in [24] proposed a scenario in the Internet of Vehicles, where game-theoretic methods are used to determine the optimal task offloading strategies for users and the optimal pricing strategies for servers during the task offloading interaction process, thereby optimizing the overall performance of both servers and users. Similarly, [25] introduced a novel MEC system comprising multiple ground servers and an aerial server carried by a UAV, with the objective of maximizing the system's weighted computational efficiency under the constraints of communication and computational resources, minimum computational requirements, and UAV mobility, thereby enhancing the system's overall performance. Fan et al. in [26] presented a game-theoretic computation task offloading strategy for IoT mobile devices, which effectively reduces the overall computation delay and energy consumption by distributing tasks among MEC BSs to balance the load, thereby improving the system's overall utility. Chen et al. in [27] proposed a two-stage Stackelberg game model for computational resource pricing and task offloading, which effectively reduces costs, latency, and energy usage by attracting idle servers to process computation tasks, thus improving the task completion rate.

Different from existing research, this study investigates the allocation of offloading destinations for EUs and the problem of task offloading and server resource pricing in a drone-assisted MEC network. First, a scenario is constructed comprising a BS edge computing server, a drone edge computing server, and multiple EUs. The interaction between servers and users is studied with the aim of optimizing both parties' utility. Furthermore, for the utility of users and servers, reward and punishment functions, as well as factors such as delay and energy consumption, are considered. For users and the initial stage of the research, game-theoretic methods are employed to analyze the user allocation problem, and a game-based GBUA algorithm is proposed to determine the target destination for EUs' task offloading. After resolving the user allocation problem, two Stackelberg game models are established, with edge servers acting as leaders to formulate pricing strategies, and users acting as followers to determine their own offloading strategies. Moreover, an iterative RPATO algorithm is proposed to optimize users' offloading strategies and server pricing strategies. Finally, the effectiveness

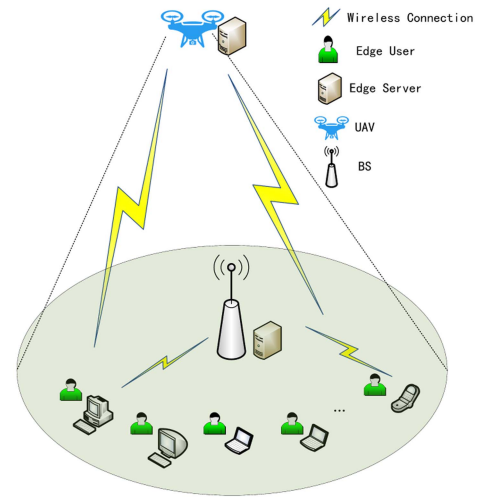


Fig. 1. Network model.

of the GBUA algorithm and RPATO algorithm is validated through simulation experiments.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Network Model

This paper investigates the task offloading and resource pricing problem in UAV-assisted MEC. The corresponding network model is depicted in Fig. 1. Which includes a UAV, a BS, and N EUs, with both the base BS and the UAV equipped with MEC servers to provide edge computing services to the EUs. The set of EUs is represented by $\mathcal{N} = \{1, 2, 3, \dots, N\}$. Each EU needs to process tasks that are either latency-sensitive or computationally intensive. Consequently, these resource-constrained EUs can choose to process their computational workload to either the BS-ES or the UAV-ES for execution in order to alleviate their computation burden. Meanwhile, according to reference [28], in the scenario of UAV-assisted multi-access edge computing, the UAV can always be in the current optimal position through trajectory optimization. Therefore, in our study, we assume that the UAV is always in the optimal position within the current environment.

Each EU generates a computation task represented as $H_i = (P_i, X_i, t_i^{\max})$, where P_i denotes the computational workload (cycles) corresponding to the size of the generated computation task, X_i represents the size of the computation task generated by the EU (MB), and $t_i^{\max}$ indicates the maximum tolerable delay (seconds) for EU i . Concurrently, each EU also decides on an offloading task represented as $h_i = (p_i, x_i, t_i^{\max})$, where p_i represents the computational workload corresponding to the size of the offloading task, x_i represents the size of the computation task offloaded to the server. Similarly, $t_i^{\max}$ represents the maximum tolerable delay for EU i . Similar to [29], different programs may require varying CPU cycles for computation. Hence, in order to quantify the number of CPU cycles required for the computation of a single bit of data, we introduce a factor ∂ . Consequently, we can obtain $p_i = x_i \times \partial$. The primary symbols employed throughout this manuscript are detailed in Table I.

TABLE I
KEY SYMBOLS

Symbol	Definition
$\mathcal{N}$	Set of EUs
$\mathcal{N}_{bs}$	Set of EUs Offloading to BS-ES
$\mathcal{N}_{uav}$	Set of EUs Offloading to UAV-ES
p_i	Computational Load of the Offloaded Task
x_i	Size of the Offloaded Task
t_i^{max}	Maximum Tolerable Latency of EUs
∂	Complexity Factor
r_i^{bs}	Transmission Rate between EU and BS-ES
f_{bs}	Computational Capacity of the BS-ES
B_{bs}	Channel Bandwidth between EU and BS-ES
h_i^{bs}	Channel Gain between EU and BS-ES
w_{bs}	Background Noise Power between EU and BS-ES
r_i^{uav}	Transmission Rate between EU and UAV-ES
f_{uav}	Computational Capacity of the UAV-ES
B_{uav}	Channel Bandwidth between EU and UAV-ES
h_i^{uav}	Channel Gain between EU and UAV-ES
w_{uav}	Background Noise Power between EU and UAV-ES
P_{hov}	Hover Energy Consumption of UAV
ℓ	Power of UAV
α	Satisfaction Factor
d_i	Price of Server Computing Resources
ν	UAV's Maximum Energy Budget

B. Communication Model

Here, let $D(y_i, \mu)$ represent a decision function for the offloading destination of EU i , which is expressed by

$$D(y_i, \mu) = \begin{cases} 1, & \text{if } y_i = \mu, \\ 0, & \text{otherwise,} \end{cases} \quad (1)$$

where y_i represents the offloading destination decision for EU i . Additionally, we introduce a parameter μ to serve as an address code for the servers when EUs make their selection. Specifically, when the server is the BS-ES, $\mu = 1$, and when it is the UAV-ES, $\mu = 0$. This implies that for EU i , $D(y_i, 1) = 1$ signifies that the computational task of EU i has been offloaded to the BS-ES, whereas $D(y_i, 0) = 1$ indicates that EU i has offloaded the computational task to the UAV-ES.

We express the transmission rate between the EU i and BS-ES as r_i^{bs} . Therefore, according to Shannon's theorem [30], r_i^{bs} can be defined as

$$r_i^{bs} = B_b \log_2 \left(1 + \frac{\varepsilon_i^{TX} h_i^{bs}}{w_{bs}} \right), \quad (2)$$

where B_{bs} indicates the bandwidth of the channel linking the EU and the BS-ES, ε_i^{TX} is indicative of the transmit power of EU i , h_i^{bs} connotes the channel coefficient between EU i and the BS-ES, w_{bs} represents background interference power.

Similarly, We express the transmission rate between the EU i and UAV-ES as r_i^{uav} . Therefore, r_i^{uav} can be defined as

$$r_i^{uav} = B_{uav} \log_2 \left(1 + \frac{\varepsilon_i^{TX} h_i^{uav}}{w_{uav}} \right), \quad (3)$$

where B_{uav} indicates the bandwidth of the channel linking the EU and the UAV-ES, h_i^{uav} connotes the channel coefficient between EU i and the UAV-ES, w_{uav} represents background interference power.

According to the transmission rate r_i^{bs} , the transmission latency when EU i offloads to the BS-ES, i.e., when $D(y_i, 1) = 1$ is given by

$$t_i^{tran,bs} = \frac{x_i}{r_i^{bs}}. \quad (4)$$

Likewise, according to the transmission rate r_i^{uav} , when EU i offloads to the UAV-ES, i.e., when $D(y_i, 0) = 1$, the transmission latency is given by

$$t_i^{tran,uav} = \frac{x_i}{r_i^{uav}}. \quad (5)$$

C. Computation Model

To simplify the computational model, this study evenly distributes the computational capacity of the BS-ES among all EUs offloading to the BS-ES concurrently. Therefore, the execution latency of EU i computational task on the BS-ES is given by

$$t_i^{com,bs} = \frac{p_i M_{bs}}{f_{bs}}, \quad (6)$$

where f_{bs} represents the computational capacity of the BS-ES (Hz), and M_{bs} denotes the aggregate tasks received by the BS-ES at the same time. Additionally, based on [31], the downstream transmission delay is negligible. Hence, the latency for EU i to offload and execute the task on the BS-ES is given by

$$t_i^{bs} = t_i^{com,bs} + t_i^{tran,bs} = \frac{x_i}{r_i^{bs}} + \frac{p_i M_{bs}}{f_{bs}}. \quad (7)$$

To compute the transmission energy consumption for EU i , this study assumes that the transmission power of the EU remains constant. Thus, the transmission energy consumption for EU i during task offloading is given by

$$E_i^{tran,bs} = t_i^{tran,bs} \varepsilon_i^{TX}. \quad (8)$$

Simultaneously, the computational energy consumption incurred by the BS-ES while processing the computational task of EU i can be expressed as

$$E_i^{bs} = k_{bs} t_i^{com,bs}, \quad (9)$$

where k_{bs} represents the CPU power consumption of the BS-ES in watts (W), being determined by the chip architecture and is considered a constant factor [32].

Similarly, the UAV-ES evenly allocates its computational capacity among all EUs offloading to it concurrently. Consequently, the execution latency of EU i 's computational task on the UAV-ES is defined as

$$t_i^{com,uav} = \frac{p_i M_{uav}}{f_{uav}}, \quad (10)$$

with f_{uav} is the computational capacity of the UAV-ES (Hz), and M_{uav} denotes the aggregate tasks received by the UAV-ES at the same time. Additionally, based on [31], the downstream

transmission delay is negligible. Hence, the latency for EU i to offload and execute the task on the UAV-ES is given by

$$t_i^{uav} = t_i^{com,uav} + t_i^{tran,uav} = \frac{x_i}{r_i^{uav}} + \frac{p_i M_{uav}}{f_{uav}}. \quad (11)$$

Simultaneously, the transmission energy consumption incurred by EU i during task offloading can be expressed as

$$E_i^{tran,uav} = t_i^{tran,uav} \varepsilon_i^{TX}. \quad (12)$$

Simultaneously, the computational energy consumption incurred from the UAV-ES as processing the computational task of EU i is expressed as

$$E_i^{com,uav} = k_{uav} t_i^{com,uav}, \quad (13)$$

where k_{uav} represents the CPU power consumption of the UAV-ES in watts (W), which is determined by the chip architecture and is considered a constant factor. Due to the limited battery budget carried by the UAV, we should consider the energy consumption of the drone while hovering in the air. According to [33], the hovering energy consumption of the UAV is given by

$$E_{hov} = \frac{P_{hov}}{\ell}, \quad (14)$$

where P_{hov} signifies the baseline hover power for the UAV, ℓ represents the energy conversion effectiveness of the UAV. Therefore, when EU i offloads the task to the UAV-ES, the total energy consumption incurred by the UAV-ES in processing this task is given by

$$E_i^{uav} = E_i^{com,uav} + E_{hov} = k_{uav} t_i^{com,uav} + \frac{P_{hov}}{\ell}. \quad (15)$$

D. Utility Function

1) *User Utility*: When considering EU utility, the logarithmic function can effectively reflect the EU's satisfaction with offloading their computational tasks, as discussed in [34], the satisfaction function can be represented as

$$F_i = \alpha \ln(1 + x_i), \quad (16)$$

where α represents the satisfaction factor of EU i , and x_i denotes the volume of the task that EU i offloads. For EUs, there exists a maximum tolerable latency. Given that the computational capacity of servers significantly exceeds that of local devices, EUs are inclined to offload tasks to servers for execution. To address the maximum tolerable latency of EUs, a reward-penalty function is introduced [35]. If the total latency incurred by offloading tasks to the server exceeds the maximum tolerable latency, a penalty is imposed on the server. Conversely, if the total latency is less than the maximum tolerable latency, a reward is given to the server. The specifics of this reward-penalty mechanism will be reflected in the utility functions of both EUs and servers. The reward-penalty function is expressed as

$$U_i^R = W_i (t_i - t_i^{\max}), \quad (17)$$

where W_i represents the reward-penalty cost coefficient for EU i . When EU offloads to the BS-ES, $t_i = t_i^{bs}$, and conversely, when offloading to the UAV-ES, $t_i = t_i^{uav}$. For the reward-penalty function, rewards are given when $t_i - t_i^{\max} < 0$, and penalties are imposed when $t_i - t_i^{\max} > 0$. Given that a EU can only offload tasks to one server, the total latency incurred by offloading to the server is uniformly represented here. Similarly, some

parameters in the EU's utility function are also consistently represented. The specific differences will be discussed separately in subsequent analyses.

Simultaneously, when EUs offload computational tasks, they incur a cost to the servers, while the servers also charge the EUs for their services. Therefore, the cost payment from EUs to the servers can be expressed as

$$C_i = p_i d_i, \quad (18)$$

where d_i represents the price at which the server sells computational resources, i.e., the price per CPU cycle. For the EU's utility function, considering the satisfaction function, reward-penalty function, transmission energy consumption, and payment cost, the utility function for the EU i is given by

$$U_i = F_i - E_i^{tran} - C_i + U_i^R. \quad (19)$$

2) *BS-ES Utility*: In this part, we will discuss BS-ES utility. For EUs offloading tasks to the BS-ES, there are inevitable costs incurred while the BS-ES executes these offloaded tasks. BS-ES utility considers three aspects, revenue from selling computational resources, energy consumption incurred by executing the offloaded tasks, and the reward-penalty function. Thus, the utility of the BS-ES is given by

$$U_{bs} = \sum_{i=1}^N D(y_i, 1) (C_i - E_i^{bs} - U_i^R). \quad (20)$$

3) *UAV-ES Utility*: For the UAV-ES utility, similar to the BS-ES, three aspects are considered, revenue from selling computational resources, energy consumption incurred by executing the offloaded tasks, and the reward-penalty function. Thus, the utility of the UAV-ES is given by

$$U_{uav} = \sum_{i=0}^N D(y_i, 0) (C_i - E_i^{uav} - U_i^R). \quad (21)$$

E. Problem Formulation

In this paper, our optimization objective is to enhance the utility for both EUs and servers. The servers generate revenue through the provision of computational resources to EUs offloading tasks to them, thereby securing a certain profit. However, the servers also incur energy consumption due to the computation of the tasks from the EU. Consequently, the problem is given by

$$\begin{aligned} \mathcal{P1} : \quad & \max U_{bs} + U_{uav}, \\ \text{s.t.} \quad & C1 : d_i^{\min} \leq d_i \leq d_i^{\max}, \\ & C2 : t_i \leq t_i^{\max}, \\ & C3 : \sum_{i=1}^{N_{uav}} E_i^{uav} \leq \nu. \end{aligned} \quad (22)$$

In the optimization problem for the servers, the utility of the servers comprises BS-ES utility and UAV-ES utility. EUs decide which server to offload their computational tasks to through an EU allocation process, which will be detailed in the subsequent sections. Therefore, in the server optimization problem, C1 states that the server's pricing for EU i must be within a threshold range acceptable to EU i . This aspect will also be elaborated upon in the following sections. C2 represents the maximum tolerable latency constraint for the EU. C3 indicates that the total computational energy consumption for tasks offloaded to

the UAV-ES should not exceed its maximum battery capacity, denoted as ν .

After presenting the optimization problem for the servers, the EU's problem is given by

$$\begin{aligned} \mathcal{P}2 : \quad & \max U_i, \\ \text{s.t.} \quad & C1 : 0 \leq p_i \leq P_i, \\ & C2 : t_i \leq t_i^{\max}, \end{aligned} \quad (23)$$

where $C1$ represents the constraint that the computational load p_i offloaded by EU i should not exceed its maximum computational task load P_i , and the second constraint represents the EU's maximum tolerable latency.

IV. GAME-BASED TASK OFFLOADING AND RESOURCE PRICING ALGORITHM DESIGN

A. Game-Based User Allocation Algorithm Design

Prior to EUs engaging in offloading operations, it is essential to ascertain the server selected by each EU. The first objective of the user allocation process is to determine the offloading destinations for each EU and preliminarily allocate resources from both the BS-ES and the UAV-ES. On the other hand, the user allocation process aims to fully utilize the computational resources of both the BS-ES and the UAV-ES, preventing the problem of resource wastage due to overloading a single server. According to the [32], the total energy consumption of the server is a significant indicator of the server's utility. Therefore, we address the issue of user server selection by considering the perspective of server energy consumption. Subsequently, we establish a model for the server's energy consumption and then propose a Game-based User Allocation (GBUA) algorithm to address this problem.

1) *User Allocation Game Formulation:* In the user allocation process, we consider that each EU conducts complete offloading operations, i.e., for each EU i , $x_i = X_i$. In this part, we aim to minimize the energy consumption of the servers and seek a strategy for user allocation. According to (9) and (15), we can obtain that

$$\begin{aligned} E(Y) &= E^b + E^u \\ &= \sum_{i=1}^n D(y_i, 1)E_i^{bs} + \sum_{i=1}^n D(y_i, 0)E_i^{uav} \\ &= \sum_{j=0}^1 \sum_{i=1}^n D(y_i, j)E_i^{b/u, j}. \end{aligned} \quad (24)$$

Based on (24), the user allocation problem is given by

$$\begin{aligned} \min \quad & E(Z), \\ \text{s.t.} \quad & C1 : y_i \in \{0, 1\}, \\ & C2 : \sum_{j=0}^1 D(y_i, j) = 1, \\ & C3 : E^u < \nu. \end{aligned} \quad (25)$$

In (25), $C1$ represents the set of potential destinations that a EU may choose, and $C2$ signifies that each EU's offloaded task can

only be offloaded to a single server. Constraint $C3$ indicates the energy consumption threshold of the UAV, denoted as ν .

In this part, we employ game theory to address the user allocation problem. We define the engagement process among EUs as a non-cooperative game, where each EU acts as a relatively independent entity. In such a game, EUs are the participants, and their decisions on selecting servers constitute the set of strategy profiles. Additionally, we can represent (24) as

$$\begin{aligned} E(Y) &= E_1(y_1, y_{-1}) + \cdots + E_i(y_i, y_{-i}) + \cdots + E_n(y_n, y_{-n}) \\ &= \sum_{j=0}^1 D(y_1, j)E_1^j + \cdots + \sum_{j=0}^1 D(y_n, j)E_n^j \\ &= D(y_1, 0)E_1^0 + D(y_1, 1)E_1^1 + \\ &\quad \cdots + D(y_n, 0)E_n^0 + D(y_n, 1)E_n^1, \end{aligned} \quad (26)$$

here, $E_i(y_i, y_{-i})$ represents the utility of EU i , with y_i representing the offloading strategy of EU i and y_{-i} indicating the offloading strategies of all other EUs except EU i . Consequently, the strategy formulation for the game in the user allocation process can be represented as $\Phi = \{(1, 2, \dots, N), (y_1, y_2, \dots, y_N), (E_1, E_2, \dots, E_N)\}$.

2) *Analysis of Nash Equilibrium for User Allocation Game:* After presenting the strategy formulation for the game, the subsequent question is whether a NE exists in this game. According to the [36], the following two lemmas are provided

Lemma 1: For a game process, if the number of participants in the game is finite and the decisions made by each participant are also finite, then the game is guaranteed to have a NE.

Theorem 1: The non-cooperative game among EUs possesses a NE.

Proof: In the game model of EUs, the number of EUs is relatively limited, and for each EU, the offloading destination can only be the BS-ES or the UAV-ES. Therefore, for each EU, the number of decisions is finite. Consequently, according to Lemma 1, a NE exists in the game among EUs. Thus, the proof of Theorem 1 is completed. $\square$

3) *Game-Based User Allocation Algorithm:* After proving the existence of a NE in the game among EUs, we introduce an iterative search algorithm, named GBUA algorithm, to address the user allocation problem. The details of this algorithm are described in Algorithm 1.

In Algorithm 1, we first initialize parameters for both the BS-ES and UAV-ES. Subsequently, the offloading destination strategy for each EU is randomly initialized. After completing these initialization steps, all EUs compete for the opportunity to update their offloading destination strategy. Suppose EU wins the update opportunity and the total server energy consumption corresponding to the alternative destination strategy is lower than that of the current strategy. In that case, the offloading destination strategy is updated. After a period of iteration, we can observe that the offloading strategy Y^* of the EUs has no new adjustments, and correspondingly, the server energy consumption does not change anymore. At this point, a NE is reached, and we obtain the offloading strategy Y^* and the corresponding energy consumption $E_{\min}$ under the NE state.

Algorithm 1: Game-based User Allocation (GBUA) Algorithm.

Input: $\mathcal{N}$, f_{bs} , f_{uav} , P_{hov} , ℓ , ν , k_{bs} , k_{uav} .

Output: $E_{\min}$, Y^* .

```

1: Initialization: Randomly initialize the EU's offloading
   strategy  $Y^*$  and calculate the corresponding server
   energy consumption  $E(Y^*)$  under this strategy.
2: repeat
3:   Obtain the load condition of the servers.
4:   EUs compete for the opportunity to update their
   offloading destinations.
5:   if EU $i$  obtains the opportunity to update their
   offloading strategy then
6:     Calculate the server energy consumption  $E^{new}$ 
   after updating the offloading strategy.
7:     if  $E^{new} < E(Y^*)$  then
8:       Update the offloading strategy and record the
   energy consumption corresponding to the updated
   offloading strategy.
9:   else
10:    Without altering the offloading strategy for
   EU $i$ .
11:   end if
12: else
13:   Without altering the offloading strategy for
   EU $i$ .
14: end if
15: until a NE is reached among the EUs.
16: Output  $Z^*$  and  $E_{\min} \leftarrow E(Y^*)$ .

```

B. Game-Based Resource Pricing and Task Offloading Algorithm Design

In this section, we employ the Stackelberg game to address the engagement problem between EUs and servers. In a Stackelberg game, participants are divided into two categories: leaders and followers. First, the leader makes their decision, and subsequently, the followers make their decisions based on the leader's announced choice. In the engagement problem between EUs and servers, the server acts as the leader while the EU acts as the follower. As mentioned earlier in the user allocation process, the EU has already determined the server. Therefore, we model the BS-ES and EUs's engagement using a Stackelberg framework, and similarly, we also model the UAV-ES and EUs's engagement using a Stackelberg approach.

1) *Stackelberg Game Formulation:* The EUs prepared to offload tasks to the BS-ES are defined as the set $\mathcal{N}_{bs} = \{1, 2, \dots, N_{bs}\}$. We consider the game between the BS-ES and the EUs to be a single-leader, multiple-followers Stackelberg model. In such a model, the BS-ES, acting as the leader, provides its pricing strategy d_i (the price for a single CPU cycle), while the followers EU i determine the size of the tasks to be offloaded to the server, denoted by p_i . Thus, the pricing strategy model and the offloading strategy model can be represented as $d = \{d_1, d_2, \dots, d_{N_{bs}}\}$ and $p = \{p_1, p_2, \dots, p_{N_{bs}}\}$ respectively. Therefore, the Stackelberg game model between the BS-ES and the EUs is formulated as

$$\Psi_1 = \{(BS - ES, EUs), (d_i, p_i), (U_{bs}, U_i)\}. \quad (27)$$

Player	BS-ES (leader)	EU i (follower)
Auction	d_i	p_i
Utility	U_{bs}	U_i

Similarly, the set of EUs prepared to offload tasks to the UAV-ES is defined as $\mathcal{N}_{uav} = \{1, 2, \dots, N_{uav}\}$, analogous to the game between the BS-ES and the EUs, the UAV-ES also provides its pricing strategy d_i . Therefore, the pricing strategy model and the offloading strategy model can be represented as $d = \{d_1, d_2, \dots, d_{N_{uav}}\}$ and $p = \{p_1, p_2, \dots, p_{N_{uav}}\}$ respectively. Thus, the Stackelberg game model between the UAV-ES and the EUs is formulated as

$$\Psi_2 = \{(UAV - ES, EUs), (d_i, p_i), (U_{uav}, U_i)\}. \quad (28)$$

Player	UAV-ES (leader)	EU i (follower)
Auction	d_i	p_i
Utility	U_{uav}	U_i

The two Stackelberg models have been fully established up to this point. The next crucial step is to prove the existence of SE for both models.

2) *Analysis of Stackelberg Equilibrium:* After the user allocation process, the EUs are divided into those offloading tasks to the BS-ES and those offloading to the UAV-ES. Based on this, two Stackelberg game models have been established. Therefore, we divide the utility optimization problem of the servers into two parts. In the first part, we optimized the utility of the BS-ES, and in the second part, we optimized the utility of the UAV-ES. The backward induction method is well-suited for dealing with the engagement between leaders and followers in Stackelberg games. Therefore, in this part, we employ this method to investigate the two Stackelberg game problems. First, each EU will provide an optimal offloading strategy based on the current state, after which the BS-ES receives the offloading strategies from each EU and responds with its own optimal pricing strategy given the current state.

Definition 1: Among the EUs offloading tasks to the BS-ES, there exists a unique NE, denoted as $p^* = (p_1^*, p_2^*, \dots, p_{N_{bs}}^*)$. Under such a strategy, for each EU's utility function, it holds that $U_i(p_i^*, p_{-i}^*) \geq U_i(p_i, p_{-i}^*)$, where p_{-i}^* represents the optimal offloading strategy for the EUs in the $\mathcal{N}_{bs}$, excluding EU i .

When the game among the EUs reaches a NE, no EU can unilaterally change their strategy to increase their utility.

Theorem 2: In the game between the BS-ES and the EUs, the BS-ES acts as the leader, and the EUs act as the followers. For each EU, there exists an optimal offloading strategy p_i^* , which can be expressed as

$$p_i^* = \frac{\alpha}{A_i + d_i - B_i} - \partial, \quad (29)$$

where

$$A_i = \frac{\varepsilon_i^{TX}}{r_i^{bs} \partial}, \quad (30)$$

$$B_i = W_i \left(\frac{1}{r_i^{bs} \partial} + \frac{M_{bs}}{f_{bs}} \right). \quad (31)$$

Proof. For EU i , its utility function satisfies (19)

$$\begin{aligned} U_i &= F_i - E_i^{\text{tran}} - p_i d_i + U_i^R \\ &= \alpha \ln(1 + x_i) - \frac{x_i}{r_i^{\text{bs}}} \varepsilon_i^{\text{TX}} - p_i d_i + \\ &\quad W_i \left(\frac{x_i}{r_i^{\text{bs}}} + \frac{p_i M_{\text{bs}}}{f_{\text{bs}}} - t_i^{\text{max}} \right). \end{aligned} \quad (32)$$

Taking the first-order derivative with respect to the offloading decision variable p_i , we get

$$\frac{\partial U_i}{\partial p_i} = \frac{\alpha}{p_i + \partial} - \frac{\varepsilon_i^{\text{TX}}}{r_i^{\text{bs}} \partial} - d_i + W_i \left(\frac{1}{r_i^{\text{bs}} \partial} + \frac{M_{\text{bs}}}{f_{\text{bs}}} \right). \quad (33)$$

Taking the second-order derivative with respect to the offloading decision variable p_i , we get

$$\frac{\partial^2 U_i}{\partial p_i^2} = -\frac{\alpha}{(p_i + \partial)^2}. \quad (34)$$

Due to $\alpha > 0$ and $(p_i + \partial)^2 > 0$, the second derivative of the EU's utility function is less than 0. Therefore, the EU's utility function is strictly concave. This proves a NE exists among the EUs. Thus, according to (33), setting $\frac{\partial U_i}{\partial p_i} = 0$. Then, we can obtain the optimal offloading strategy for EU i , which is (29). $\square$

Thus, Theorem 2 is proven. Next, by setting the offloading strategy $p_i^* = 0$ and $p_i^* = P_i$, we can derive the minimum and maximum pricing strategies that EU i can accept, represented as

$$d_i^{\min} = \frac{\alpha}{\partial + P_i} + B_i - A_i, \quad (35)$$

$$d_i^{\max} = \frac{\alpha}{\partial} + B_i - A_i. \quad (36)$$

The optimal pricing strategy for the BS-ES should lie between $d_i^{\min}$ and $d_i^{\max}$. Therefore, the optimal offloading strategy for the EU can be expressed as

$$p_i^* = \begin{cases} 0, & d_i \geq d_i^{\max}, \\ \frac{\alpha}{A_i + d_i - B_i} - \partial, & d_i^{\min} < d_i < d_i^{\max}, \\ P_i, & d_i \leq d_i^{\min}. \end{cases} \quad (37)$$

Lemma 2: Each EU has a unique optimal offloading strategy for a given pricing strategy of the BS-ES.

Proof: For EUs, when the server's pricing is too high, i.e., $d_i \geq d_i^{\max}$, EUs will not offload to the BS-ES. When the BS-ES's pricing is too low, i.e., $d_i \leq d_i^{\min}$, EUs will choose to offload all tasks to the edge server. However, when $d_i^{\min} < d_i < d_i^{\max}$, it can be derived that

$$\frac{\partial p_i^*}{\partial d_i} = -\frac{\alpha}{(A_i + d_i - B_i)^2}, \quad (38)$$

$$\frac{\partial^2 p_i^*}{\partial d_i^2} = \frac{2\alpha}{(A_i + d_i - B_i)^3}. \quad (39)$$

It can be deduced that (38) is less than 0, while (39) is greater than 0. This implies that when the pricing of the BS-ES falls within the maximum and minimum acceptable ranges for the EU, there is an inverse proportional relationship between the EU's offloading quantity and the BS-ES's pricing.

In summary, we can conclude that under the given BS-ES pricing strategy, each EU has a unique optimal offloading strategy. $\square$

Definition 2: If $U_{\text{bs}}(d_i^*, p_i^*) \geq U_{\text{bs}}(d_i, p_i^*)$, then a unique SE exists between the BS-ES and the EUs.

Theorem 3: Considering the optimal task offloading strategy for all EUs offloading to the BS-ES, the BS-ES has a unique

optimal pricing strategy. Based on this, the BS-ES and the EUs establish a unique SE.

Proof: Substituting the optimal task offloading strategy for EUs into the BS-ES's utility function, (20), and taking the partial derivative with respect to the pricing strategy d_i , we get

$$\frac{\partial U_{\text{bs}}}{\partial d_i} = \frac{\partial p_i^*}{\partial d_i} \left(d_i - k_{\text{bs}} \frac{M_{\text{bs}}}{f_{\text{bs}}} - B_i \right) + p_i^*. \quad (40)$$

Thus, the second partial derivative with respect to d_i can be expressed as

$$\begin{aligned} \frac{\partial^2 U_{\text{bs}}}{\partial d_i^2} &= \frac{\partial^2 p_i^*}{\partial d_i^2} \left(d_i - k_{\text{bs}} \frac{M_{\text{bs}}}{f_{\text{bs}}} - B_i \right) + \frac{\partial p_i^*}{\partial d_i} + \frac{\partial p_i^*}{\partial d_i} \\ &= \frac{-2\alpha \left(k_{\text{bs}} \frac{M_{\text{bs}}}{f_{\text{bs}}} + A_i \right)}{(A_i + d_i - B_i)^3}. \end{aligned} \quad (41)$$

Given that $2\alpha(k_{\text{bs}} \frac{M_{\text{bs}}}{f_{\text{bs}}} + A_i) > 0$ and according to (35) and (36), $(A_i + d_i - B_i)^3 > 0$, we can conclude that the second partial derivative of the BS-ES's utility function with respect to d_i is negative. Hence, the utility function of the BS-ES is strictly concave, we can establish that there exists a SE between the EU and the BS-ES. $\square$

Similarly, by setting the (40) to zero, we ascertain

$$d_i^* = B_i - A_i + \sqrt{\frac{\alpha \left(k_{\text{bs}} \frac{M_{\text{bs}}}{f_{\text{bs}}} + A_i \right)}{\partial}}. \quad (42)$$

Since the optimal pricing strategy d_i^* for the BS-ES should lie between $d_i^{\min}$ and $d_i^{\max}$, the optimal pricing strategy can be expressed as

$$d_i^* = \begin{cases} d_i^{\max}, & d_i \geq d_i^{\max}, \\ B_i - A_i + \sqrt{\frac{\alpha \left(k_{\text{bs}} \frac{M_{\text{bs}}}{f_{\text{bs}}} + A_i \right)}{\partial}}, & d_i^{\min} < d_i < d_i^{\max}, \\ d_i^{\min}, & d_i \leq d_i^{\min}. \end{cases} \quad (43)$$

Having thus investigated the BS-ES and EUs's engagement, we have established the presence of a SE between them. Consequently, in this part, we examine the UAV-ES and EUs's engagement. Initially, each EU determines its optimal offloading strategy based on the current state. Subsequently, the UAV-ES receives these offloading strategies from each EU and then determines its own optimal pricing strategy given the current state.

Definition 3: Among the EUs offloading to the UAV-ES, there exists exactly one NE, denoted as $p^* = (p_1^*, p_2^*, \dots, p_{N_{\text{uav}}}^*)$. Under this strategy profile, for the utility function of each EU, it holds that $U_i(p_i^*, p_{-i}^*) \geq U_i(p_i, p_{-i}^*)$, where p_{-i}^* represents the optimal offloading strategies of all other EUs offloading to the UAV-ES except for EU i .

Concomitantly, when the game reaches NE, no EU can unilaterally adjust its strategy to increase its own utility.

Theorem 4: Taking into account the optimal offloading strategies of all EUs who offload to the UAV-ES, the UAV-ES possesses a unique optimal pricing strategy. Based on this, a unique SE between the UAV-ES and the EUs is established.

The proof process of Theorem 4 resembles that of the SE between the BS-ES and the EUs. Hence, no further elaboration is provided here.

Algorithm 2: Resource Pricing and Task Offloading (RPATO) Algorithm.

Input: $\mathcal{N}$, f_{bs} , f_{uav} , P_{hov} , ℓ , ν , k_{bs} , k_{uav} , other parameters.

Output: d^* , p^* .

- 1: **Initialization:** step size θ , precision threshold $\bar{U}$, Maximum number of iterations ψ .
 - 2: Two parameters, u and v , are set to record the number of iterations.
 - 3: Set the initial pricing strategy for each EU such that it actually satisfies the condition between the maximum and minimum values.
 - 4: The server updates its pricing strategy according to the gradient-based optimization method.
 - 5: After the server completes an update cycle of its pricing strategy, each EU formulates its own offloading strategy based on the current state.
 - 6: Each EU updates its offloading strategy according to the gradient-based optimization method.
 - 7: **while** $u < \psi$ **do**
 - 8: At each pass of iteration u
 - 9: Do (2) and (3)
 - 10: **while** $n < \psi$ **do**
 - 11: At each pass of iteration v
 - 12: Do (4) and (5)
 - 13: $v = v + 1$
 - 14: **end while**
 - 15: $u = u + 1$
 - 16: $u = 0$
 - 17: **end while**
-

3) *Stackelberg Game-Based Resource Pricing and Task Offloading Algorithm:* Having established the existence of SE, this part introduces the RPATO algorithm to solve the games between the servers and the EUs. Through this algorithm, the optimal offloading strategies and the optimal pricing strategies are obtained. In this part, the RPATO algorithm is explained in detail based on the example of the BS-ES and the EUs' engagement.

Algorithm 2 provides a detailed description of the RPATO algorithm. Initially, the inputs include the computational capacity of the BS-ES, the set of EUs offloading to the BS-ES, and other parameters. In Line 1, the step size θ , the precision threshold $\bar{U}$, and the maximum number of iterations ψ are initialized, while setting the initial iteration counts $u = 0$ and $v = 0$. In Line 2 and Line 3, the BS-ES's pricing strategy $d(0)$ is initialized and adjusted to fall within the range of $d_i^{\min}$ and $d_i^{\max}$ according to (35) and (36). The strategy is then updated once according to the gradient-based optimization method. Concurrently, in Line 4 and Line 5, the EUs' offloading strategy $p(0)$ is initialized to lie between 0 and the EU's maximum offloading capacity P_i , and then updated once according to the gradient-based optimization method.

In the Algorithm 2, the EUs' offloading strategies and the BS-ES's pricing strategies converge to the optimal solution iteratively. During the iteration process, the server's pricing strategy is updated first, and the number of outer iterations, i.e., $m = m + 1$. Subsequently, the EUs begin their iteration

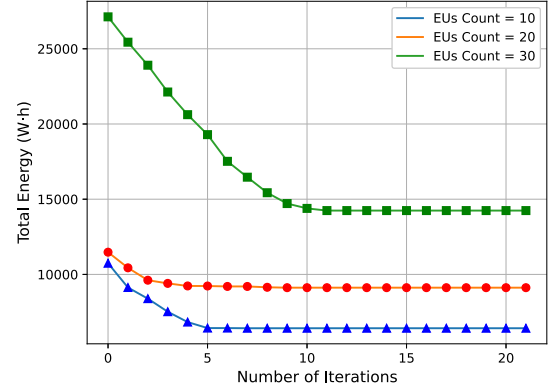


Fig. 2. Convergence analysis of GBUA algorithm.

TABLE II
EXPERIMENT PARAMETERS

Parameters	Value Settings
Number of EUs	[10, 30]
Computing task size	[10, 150] MB
Complexity factor	1600 cycles/byte
CPU power of BS-ES	[0.10, 0.15] W
CPU power of UAV-ES	[0.10, 0.15] W
Transmission power	[1.0, 3.0] W
Background noise	-100 dBm
Bandwidth of BS-ES	4 MHz
Bandwidth of UAV-ES	40 MHz

based on the current pricing strategy and stop upon achieving the upper limit of iteration counts, the process concludes. At this point, the server updates its pricing strategy based on the EUs' current optimal offloading strategies. When the server's iteration count reaches the maximum number of iterations, the iteration ceases, and an approximate optimal solution for the offloading and pricing strategies is obtained.

V. EXPERIMENTAL EVALUATION

In this section, the convergence of the GBUA and RPATO algorithms is analyzed through experimentation. Additionally, the RPATO algorithm is compared with other benchmark algorithms, and the changes in utility for EUs and servers under varying parameters are investigated.

A. Parameter Settings

In this section, the parameters for the experiment are set. According to [32], the processing capabilities of both the BS-ES and the UAV-ES are assigned random values ranging from 0.10 watts to 0.15 watts. Furthermore, based on [11], the background noise for both the BS-ES and the UAV-ES is set to -100 dBm. The transmission power of the EUs is set to a random value between 1.0W and 3.0W. Additionally, the data volume of the EUs is randomly distributed between 10 MB and 150 MB. Table II presents some of the parameters required for the experiment.

In addition, the following three benchmark methods are provided for performance comparison:

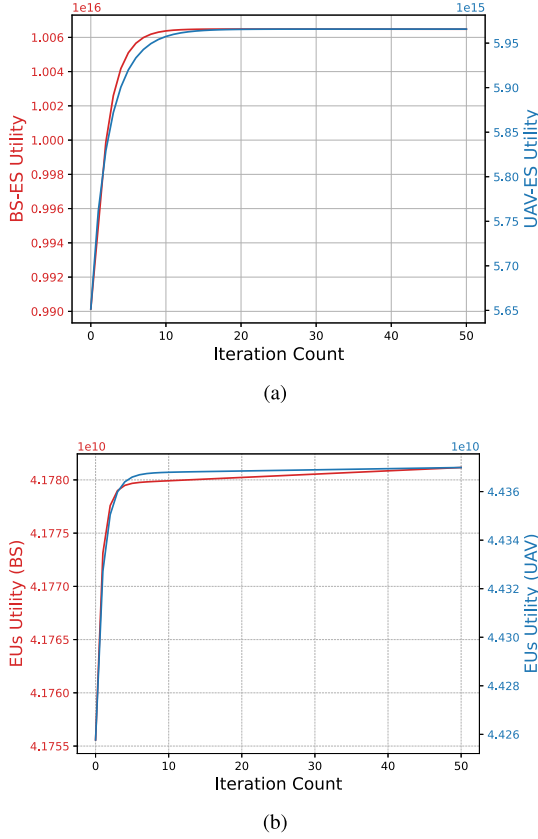


Fig. 3. Convergence analysis of RPATO algorithm. (a) ES-Utility vs. Iteration Count. (b) EUs Utility vs. Iteration Count.

- **Random Offloading Algorithm (Random):** The decisions for task offloading by EUs and pricing by servers are generated randomly.
- **Full Offloading Algorithm (FO):** EUs choose to offload all computational tasks to the server for execution, and then the server determines the optimal pricing strategy through the RPATO algorithm.
- **Uniform Pricing Algorithm (Uniform):** In the uniform pricing algorithm, the server no longer sets prices for computing resources individually for each EU but instead adopts a uniform pricing strategy. During each iteration, the offloading strategies of all EUs need to be updated, while the server's pricing strategy only needs to be updated once [34].
- **ICSOC-19:** During the iterative time slot, all EUs simultaneously update their offloading strategies based on the pricing set by the service provider [37].
- **GTCO-21:** The iterative strategy involves selecting pricing decisions from the set that can increase the total utility and enhance the server's utility [38].

B. Convergence Analysis

First, we analyze the convergence properties of the GBUA algorithm and the RPATO algorithm.

In Fig. 2, we have conducted an analysis of the GBUA algorithm. We set up three samples with varying numbers of EUs: 10, 20, and 30. From the graph, it is visually evident that before

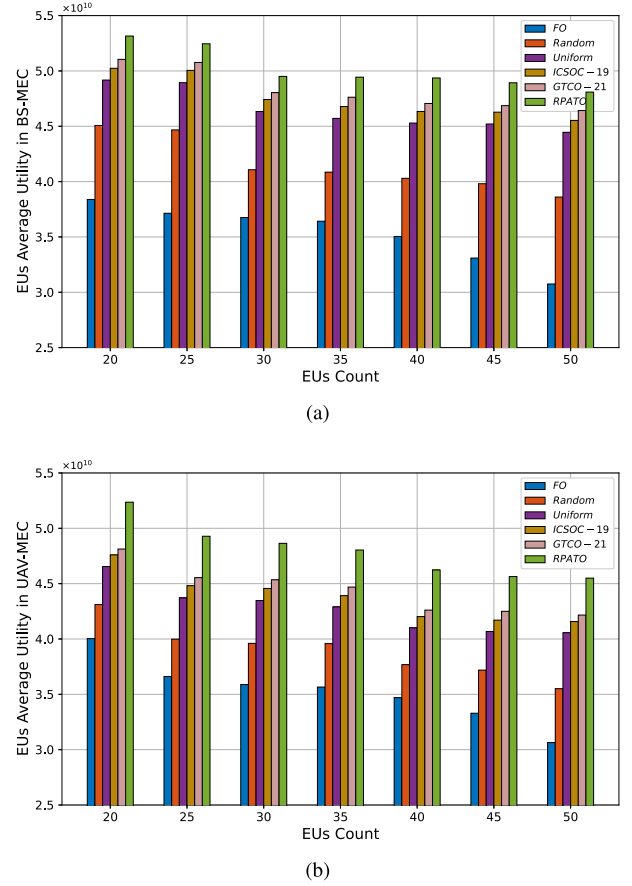


Fig. 4. The changes in the average utility of EUs at different numbers of EUs. (a) EUs Average Utility in BS-MEC vs. EUs Count. (b) EUs Average Utility in UAV-MEC vs. EUs Count.

any iterations are performed, the total energy consumption of the servers exhibits a high trend. However, after a certain number of iterations, the total energy consumption of the servers begins to decrease and eventually stabilizes.

Following the analysis of the GBUA algorithm, we proceed to investigate the convergence properties of the RPATO algorithm. In Fig. 3, we simultaneously analyze the performance of the RPATO algorithm in both the BS-ES and UAV-ES scenarios. The red curve is employed to represent the utility variations of the BS-ES and the EUs offloaded to the BS-ES server, while the blue curve indicates the utility changes of the UAV-ES and the EUs offloaded to the UAV-ES server. It can be visually observed that prior to the initiation of iterations, the utility of the servers and the average utility of the EUs are relatively low. However, as the number of iterations increases, there is a significant rise in utility, which approaches a stable state after a certain number of iterations.

C. Performance Comparison

After analyzing the convergence of the GBUA and RPATO algorithms, we introduced three additional algorithms to compare with the RPATO algorithm, aiming to assess the performance of RPATO. Simultaneously, we adjusted the values of several parameters and analyzed how these changes would affect the performance of the RPATO algorithm.

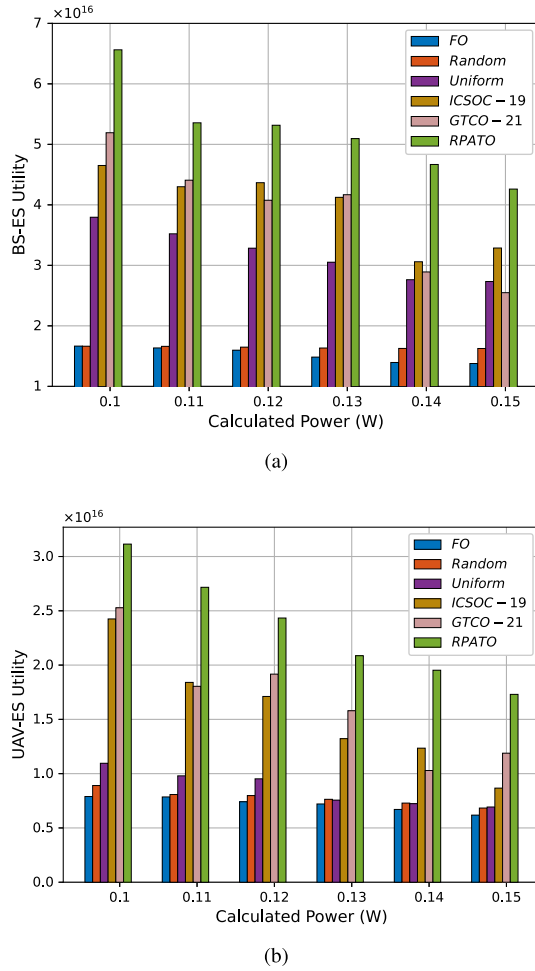


Fig. 5. Analysis of the utility change for servers with varying computational power. (a) BS-ES Utility vs. Calculated Power (W). (b) UAV-ES Utility vs. Calculated Power (W).

First, we analyzed the parameter of the number of users. Similarly, we separately examined the impact of variations in the number of users in the BS scenario and the UAV scenario. In Fig. 4, we analyzed the user utility when the number of users changes in both the BS and UAV scenarios. It can be observed that, under varying numbers of EUs, the RPATO algorithm exhibits superior performance compared to the other five benchmark algorithms. Additionally, the RPATO algorithm shows a decreasing trend as the number of EUs increases. This is because, with an increase in the number of EUs, the computing power allocated to each EU by the server is relatively smaller, leading to a decrease in average EU utility. Similarly, the other five benchmark algorithms also exhibit the same trend as the RPATO algorithm with the increase in the number of EUs.

Fig. 5 illustrates the impact of server computing power on server utility. Fig. 5(a) shows the changes in utility of the BS-ES. Meanwhile, Fig. 5(b) depicts changes in utility of the UAV-ES. One can observe that under different computing powers, the RPATO algorithm outperforms the other five benchmark algorithms. Additionally, as the computing power of the server increases, the utility of the server shows a decreasing trend. This is because, with the increase in computing power, the energy consumption of the server in processing tasks of equal size

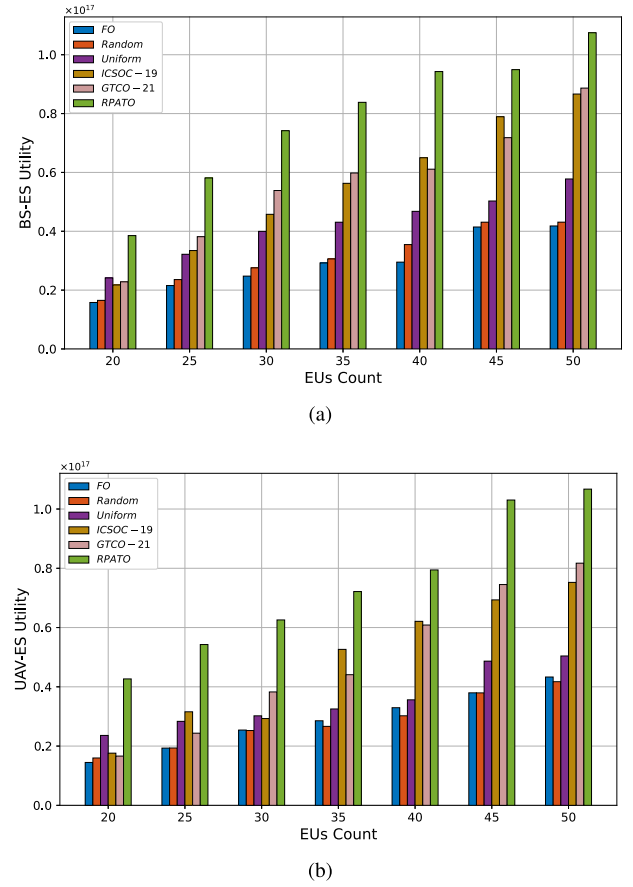


Fig. 6. Analysis of the utility change for servers with varying numbers of EUs. (a) BS-ES Utility vs. EUs Count. (b) BS-ES Utility vs. EUs Count.

also increases. Therefore, both the RPATO algorithm and the other five benchmark algorithms exhibit the same trend as the computing power of the server increases.

In Fig. 6, we analyzed the changes in server utility under varying numbers of users. We examined the utility changes for both BS-ES and UAV-ES. In Fig. 6, it is visually evident that as the number of users increases, the utility of both BS-ES and UAV-ES experiences a significant rise. Additionally, it can be noted that, the RPATO algorithm performs better than the other five benchmark algorithms. Additionally, the other five benchmark algorithms show the same trend as the RPATO algorithm with the increase in the number of EUs, i.e., the utility of the server increases with the number of EUs. This is because, with the increase in the number of EUs, the computing tasks offloaded to the server also increase, generating more revenue for the server and thus increasing its utility.

Fig. 7 demonstrates the changes in average EU utility under different transmission powers. Fig. 7(a) shows the changes in average EU utility when tasks are offloaded to the BS-ES, and Fig. 7(b) depicts the changes when tasks are offloaded to the UAV-ES. It can be observed that under different transmission powers, the RPATO algorithm performs better than the other five benchmark algorithms. Moreover, the RPATO algorithm exhibits a decreasing trend as the transmission rate increases. This is because, with the increase in transmission power, EUs need to consume more transmission energy when offloading tasks to the

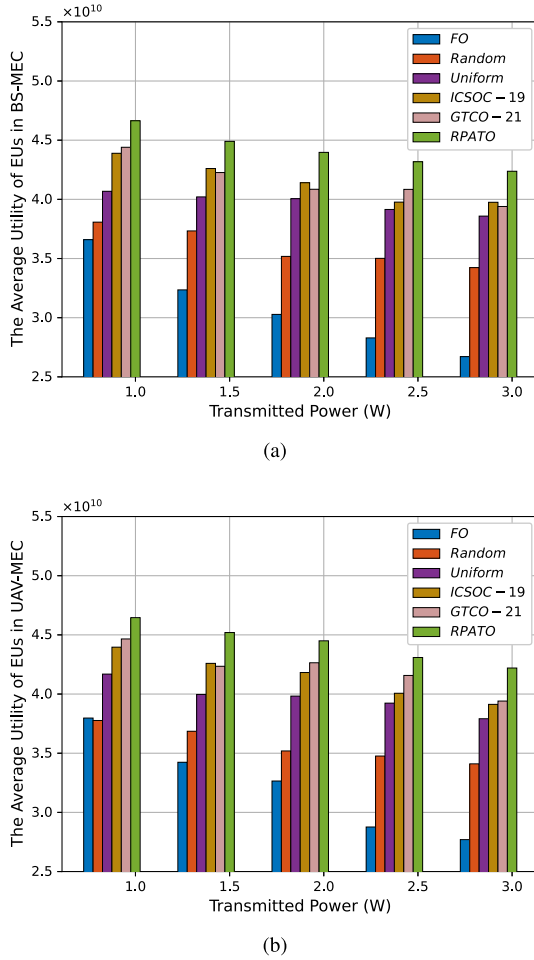


Fig. 7. The variation in the average utility of EUs at different transmission power levels. (a) The Average Utility of EUs in BS-MEC vs. Transmitted Power (W). (b) The Average Utility of EUs in UAV-MEC vs. Transmitted Power (W).

server, leading to a decrease in average EU utility. Similarly, The five algorithms we used for comparison also showed a downward trend when the transmission power changed.

VI. CONCLUSION

This paper investigates the task offloading and resource pricing problem for EUs in UAV-assisted MEC networks. Initially, a scenario is considered that involves a BS-ES, a UAV-ES, and multiple EUs. The engagement between the servers and EUs is studied with the objective of optimizing the utility for both parties. First, game theory is employed to examine the user allocation problem, and a game-based algorithm named GBUA is proposed to determine the task offloading destinations for EUs. After addressing the user allocation problem, two Stackelberg game models are established, where the edge servers act as leaders to provide pricing strategies, and the EUs as followers to determine their offloading strategies. Additionally, an iterative algorithm named RPATO is proposed to obtain the EUs' offloading strategies and the servers' pricing strategies. Finally, the effectiveness of the GBUA and RPATO algorithms is validated through simulation experiments. As future work, our objective is to investigate the problem of UAV positioning and

trajectory optimization while researching task offloading and resource allocation in the scenario of UAV-assisted multi-access edge computing.

REFERENCES

- [1] G. Cheng et al., "A lightweight authentication-driven trusted management framework for IoT collaboration," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 747–760, May/Jun. 2024.
- [2] Y. Zhang, P. Zhang, C. Jiang, S. Wang, H. Zhang, and C. Rong, "QoS aware virtual network embedding in space-air-ground-ocean integrated network," *IEEE Trans. Services Comput.*, vol. 17, no. 4, pp. 1712–1723, Jul./Aug. 2024.
- [3] K. Jiang, C. Sun, H. Zhou, X. Li, M. Dong, and V. C. M. Leung, "Intelligence-empowered mobile edge computing: Framework, issues, implementation, and outlook," *IEEE Netw.*, vol. 35, no. 5, pp. 74–82, Sep./Oct. 2021.
- [4] Z. Zhou, P. Liu, J. Feng, Y. Zhang, S. Mumtaz, and J. Rodriguez, "Computation resource allocation and task assignment optimization in vehicular fog computing: A contract-matching approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3113–3125, Apr. 2019.
- [5] K. Peng, P. Xiao, S. Wang, and V. C. Leung, "SCOF: Security-aware computation offloading using federated reinforcement learning in Industrial Internet of Things with edge computing," *IEEE Trans. Services Comput.*, vol. 17, no. 4, pp. 1780–1792, Jul./Aug. 2024.
- [6] C. Zhang, J. Yin, and S. Deng, "Ensuring fairness in edge networks: A GNN-based media workload migration scheme with fairness guarantee," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 934–948, May/Jun. 2024.
- [7] P. Zhang, H. Jin, H. Dong, W. Song, and A. Bouguettaya, "Privacy-preserving QoS forecasting in mobile edge environments," *IEEE Trans. Services Comput.*, vol. 15, no. 2, pp. 1103–1117, Mar./Apr. 2022.
- [8] Y. Xu, T. Zhang, Y. Liu, D. Yang, L. Xiao, and M. Tao, "UAV-assisted MEC networks with aerial and ground cooperation," *IEEE Trans. Wireless Commun.*, vol. 20, no. 12, pp. 7712–7727, Dec. 2021.
- [9] S. A. Huda and S. Moh, "Survey on computation offloading in UAV-enabled mobile edge computing," *J. Netw. Comput. Appl.*, vol. 201, 2022, Art. no. 103341.
- [10] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, "Energy efficient task offloading and resource allocation in air-ground integrated MEC systems: A distributed online approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 8, pp. 8129–8142, Aug. 2024.
- [11] H. Zhou, Z. Wang, G. Min, and H. Zhang, "UAV-aided computation offloading in mobile-edge computing networks: A Stackelberg game approach," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6622–6633, Apr. 2023.
- [12] X. Yuan, Z. Xie, and X. Tan, "Computation offloading in UAV-enabled edge computing: A Stackelberg game approach," *Sensors*, vol. 22, no. 10, 2022, Art. no. 3854.
- [13] Y. Xu et al., "A one-leader multi-follower Bayesian-Stackelberg game for anti-jamming transmission in UAV communication networks," *IEEE Access*, vol. 6, pp. 21 697–21 709, 2018.
- [14] Y. Yang, W. Wang, L. Liu, K. Dev, and N. M. F. Qureshi, "AoI optimization in the UAV-aided traffic monitoring network under attack: A Stackelberg game viewpoint," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 932–941, Jan. 2023.
- [15] P. Lai et al., "Dynamic user allocation in stochastic mobile edge computing systems," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 2699–2712, Sep./Oct. 2022.
- [16] K. Guo, R. Gao, W. Xia, and T. Q. S. Quek, "Online learning based computation offloading in MEC systems with communication and computation dynamics," *IEEE Trans. Commun.*, vol. 69, no. 2, pp. 1147–1162, Feb. 2021.
- [17] J. Wang, D. Feng, S. Zhang, A. Liu, and X.-G. Xia, "Joint computation offloading and resource allocation for MEC-enabled IoT systems with imperfect CSI," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3462–3475, Mar. 2021.
- [18] P. Dai, K. Hu, X. Wu, H. Xing, F. Teng, and Z. Yu, "A probabilistic approach for cooperative computation offloading in MEC-assisted vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 899–911, Feb. 2022.
- [19] N. Waqar, S. A. Hassan, A. Mahmood, K. Dev, D.-T. Do, and M. Gidlund, "Computation offloading and resource allocation in MEC-enabled integrated aerial-terrestrial vehicular networks: A reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21 478–21 491, Nov. 2022.

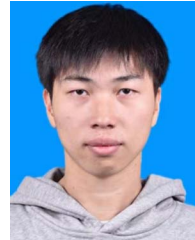
- [20] S. Zhu, L. Gui, D. Zhao, N. Cheng, Q. Zhang, and X. Lang, "Learning-based computation offloading approaches in UAVs-assisted edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 928–944, Jan. 2021.
- [21] J. Lin, L. Huang, H. Zhang, X. Yang, and P. Zhao, "A novel lyapunov based dynamic resource allocation for UAVs-assisted edge computing," *Comput. Netw.*, vol. 205, 2022, Art. no. 108710.
- [22] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.
- [23] W. Lu et al., "Secure transmission for multi-UAV-assisted mobile edge computing based on reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1270–1282, May/Jun. 2023.
- [24] X. Xu et al., "Game theory for distributed IoV task offloading with fuzzy neural network in edge computing," *IEEE Trans. Fuzzy Syst.*, vol. 30, no. 11, pp. 4593–4604, Nov. 2022.
- [25] Y. Hui, Z. Su, T. H. Luan, C. Li, G. Mao, and W. Wu, "A game theoretic scheme for collaborative vehicular task offloading in 5G HetNets," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16 044–16 056, Dec. 2020.
- [26] W. Fan, L. Yao, J. Han, F. Wu, and Y. Liu, "Game-based multitype task offloading among mobile-edge-computing-enabled base stations," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17 691–17 704, Dec. 2021.
- [27] Y. Chen, J. Wu, J. Han, H. Zhao, and S. Deng, "A game-theoretic approach based task offloading and resource pricing method for idle vehicle devices assisted VEC," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21954–21969, Jun. 2024.
- [28] D. Wang, J. Tian, H. Zhang, and D. Wu, "Task offloading and trajectory scheduling for UAV-enabled MEC networks: An optimal transport theory perspective," *IEEE Wireless Commun. Lett.*, vol. 11, no. 1, pp. 150–154, Jan. 2022.
- [29] T. Mahn and A. Klein, "Energy-efficient application-aware mobile edge computing with multiple access points," in *Proc. IEEE 31st Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2020, pp. 1–7.
- [30] A. Jerri, "The Shannon sampling theorem—Its various extensions and applications: A tutorial review," in *Proc. IEEE*, vol. 65, no. 11, pp. 1565–1596, 1977.
- [31] H. Zhou, K. Jiang, X. Liu, X. Li, and V. C. M. Leung, "Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1517–1530, Jan. 2022.
- [32] M. Wang, L. Zhang, P. Gao, X. Yang, K. Wang, and K. Yang, "Stackelberg-game-based intelligent offloading incentive mechanism for a multi-UAV-assisted mobile-edge computing system," *IEEE Internet Things J.*, vol. 10, no. 17, pp. 15 679–15 689, Sep. 2023.
- [33] J. K. Stolaroff, C. Samaras, E. R. O'Neill, A. Lubers, A. S. Mitchell, and D. Ceperley, "Energy use and life cycle greenhouse gas emissions of drones for commercial package delivery," *Nat. Commun.*, vol. 9, no. 1, 2018, Art. no. 409.
- [34] J. Zhang, X. Huang, and R. Yu, "Optimal task assignment with delay constraint for parked vehicle assisted edge computing: A Stackelberg game approach," *IEEE Commun. Lett.*, vol. 24, no. 3, pp. 598–602, Mar. 2020.
- [35] H. Zhou, Z. Wang, N. Cheng, D. Zeng, and P. Fan, "Stackelberg-game-based computation offloading method in cloud-edge computing networks," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16 510–16 520, Sep. 2022.
- [36] J. F. Nash Jr., "Equilibrium points in n-person games," in *Proc. Nat. Acad. Sci. USA*, vol. 36, no. 1, pp. 48–49, 1950.
- [37] P. Lai et al., "Edge user allocation with dynamic quality of service," in *Proc. 17th Int. Conf. Service-Oriented Comput.*, Springer, 2019, pp. 86–101.
- [38] S. Jeong, O. Simeone, and J. Kang, "Mobile edge computing via a UAV-mounted cloudlet: Optimization of bit allocation and path planning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2049–2063, Mar. 2018.



Zhuoyue Chen is currently working toward the MEng degree with the School of Computer Science, Beijing Information Science and Technology University, Beijing, China. His current research interests include edge computing and machine learning.



Yaozong Yang is currently working toward the MEng degree in computer science and technology, the Beijing Information Science and Technology University, China. His current research interests include edge computing, Internet of Things, stochastic optimization, and reinforcement learning.



Jiajie Xu is currently working toward the MEng degree with the School of Computer Science, Beijing Information Science and Technology University, Beijing, China. His current research interests include edge computing and machine learning.



Ying Chen (Senior Member, IEEE) received the PhD degree in computer science and technology from Tsinghua University, Beijing, China, in 2017. She was a joint PhD student with the University of Waterloo, Waterloo, ON, Canada from 2016 to 2017. She is a professor with the Computer School, Beijing Information Science and Technology University, Beijing. Her current research interests include Internet of Things, mobile edge computing, wireless networks and communications, machine learning, etc. She is the recipient of the Best Paper Award with IEEE SmartIoT 2019, the 2016 Google PhD Fellowship Award, and the 2014 Google Anita Borg Award, 2022 Outstanding Contribution Award in 18th EAI CollaborateCom, respectively. She is/was the leading guest editor of *Journal of Cloud Computing*, TPC member of IEEE HPCC, and PC member of IEEE Cloud, CollaborateCom, IEEE CPSCom, CSS, etc. She is also the reviewer of several journals, such as the *IEEE Wireless Communications Magazine*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Internet of Things Journal*, *IEEE Transactions on Cloud Computing*, and *IEEE Transactions on Services Computing*.



Jiwei Huang (Senior Member, IEEE) received the BEng and PhD degrees in computer science and technology from Tsinghua University in 2009 and 2014, respectively. He was a visiting scholar with the Georgia Institute of Technology. Currently, he is a professor and associate dean with the College of Artificial Intelligence, China University of Petroleum (Beijing), a member of the Hainan Institute of China University of Petroleum (Beijing), and the director of the Beijing Key Laboratory of Petroleum Data Mining. His research areas include services computing, Internet of Things, and edge computing. He has published one book and more than 70 papers in international journals and conference proceedings, including *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Vehicular Technology*, ACM SIGMETRICS, IEEE ICWS, and IEEE SCC. He currently serves on the editorial boards of the *Chinese Journal of Electronics and Scientific Programming*.